

Documentación de Interfaz Web: Comedero Pet

Análisis de la arquitectura, componentes HTML5 y lógica de control con JavaScript (index.html y script.js).

Propósito de la Aplicación Web

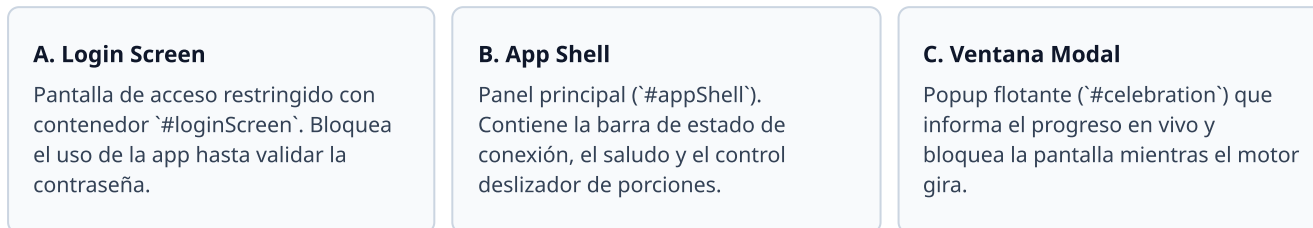
La aplicación web actúa como el Panel de Control interactivo para el usuario. Permite autenticarse mediante contraseña, seleccionar el tiempo de rotación del motor y enviar la orden de alimentación al servidor API en tiempo real.

1. Arquitectura General del Cliente Web



2. Estructura HTML del Documento (index.html)

La interfaz está organizada en tres bloques jerárquicos principales dentro del cuerpo HTML:



Elementos clave de entrada de usuario:

- `<input type="password" id="passwordInput">`: Campo de texto seguro para ingresar la clave de acceso.
- `<input type="range" id="portionRange" min="1" max="60">`: Deslizador para ajustar el tiempo de accionamiento del motor (1 a 60 segundos).
- `<button id="serveButton">`: Desencadenante principal para despachar la instrucción al servidor API.

3. Lógica de Control en JavaScript (script.js)

El código JavaScript gestiona el comportamiento interactivo, la seguridad del cliente y la sincronización con la API.

Módulo 1: Control de Autenticación y Sesión

Para evitar solicitar la clave continuamente, la aplicación almacena la validación en el almacenamiento temporal del navegador (sessionStorage).

```
const VALID_PASSWORD = 'Daas1243'; function checkAuth() { if (sessionStorage.getItem('authenticated') === 'true') { loginScreen.style.display = 'none'; appShell.style.display = 'block'; } else { loginScreen.style.display = 'flex'; appShell.style.display = 'none'; } }
```

Módulo 2: Sincronización del Deslizador (Slider)

Al mover la barra deslizante, el script captura el valor numérico y actualiza dinámicamente el texto de la interfaz y el gradiente de color CSS.

```
function updatePortion() { currentDuration = Number(portionRange.value); portionValue.textContent = currentDuration; portionHint.textContent = getDurationDescription(currentDuration); }
```

4. Consumo de Servicios REST (Endpoints)

La aplicación web realiza peticiones asíncronas HTTP hacia la API Backend utilizando la función nativa fetch():

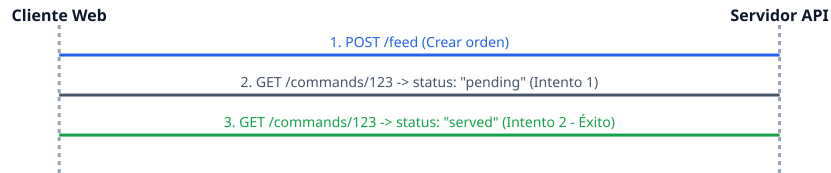
Acción Web	Método HTTP	Ruta Endpoint	Descripción Técnica
Enviar Orden	POST	/api/v1/feed	Envía el device_id y los segundos seleccionados en formato JSON.
Ver Conexión	GET	/api/v1/devices/{id}/status	Consulta cada 15 segundos si el ESP32 está en línea (Online/Offline).
Estado Orden	GET	/api/v1/commands/{id}	Consulta repetida para saber si la orden fue completada o falló.

Manejo del Estado de Conexión en la Interfaz:

La función checkDeviceStatus() consulta periódicamente el endpoint del dispositivo. Si la API responde "status": "online", se añade la clase CSS is-online (indicador verde). Si la petición falla o retorna "offline", cambia a is-offline (indicador rojo).

5. Algoritmo de Polling (Consulta Periódica)

Debido a que el protocolo HTTP no permite al servidor enviar datos sin previa solicitud del cliente, la web implementa el patrón **Polling** para monitorear el estado del motor.



Implementación del Bucle de Verificación:

```
async function checkCommandStatus(commandId) { if (pollingAttempts >= MAX_POLLING_ATTEMPTS) { celebrationTitle.textContent = 'Sin respuesta del comedero'; unlockCelebrationModal(); // Permite cerrar por tiempo agotado return; } pollingAttempts++; const response = await fetch(`${API_BASE_URL}/commands/${commandId}`); const command = await response.json(); if (command.status === 'served') { celebrationTitle.textContent = 'Porción servida con éxito'; unlockCelebrationModal(); // Habilita botones de cierre return; } // Reintenta en 3 segundos si continúa pendiente commandPollingTimer = window.setTimeout(() => checkCommandStatus(commandId), 3000); }
```

6. Secuencia Operativa Paso a Paso

Paso 1 Usuario ingresa clave en la pantalla de login.

Paso 2 Validación exitosa y guardado en sessionStorage.

Paso 3 Usuario ajusta tiempo (ej. 20 segundos) y presiona Servir.

Paso 4 Envío de petición POST /api/v1/feed.

Paso 5 Despliegue del modal e inicio de bloqueo de pantalla.

Paso 6 Bucle de consultas GET /commands/{id} cada 3s.

Paso 7 El ESP32 ejecuta la orden y actualiza a served.

Paso 8 Confirmación en la web y liberación de los botones.